Multimedia Signal processing

Prof. Marco Tagliasacchi

February, 9 2012

1. **Question (8 pts)**:
   Consider a sequence of symbols emitted by a source with memory

$$\{2, 1, 1, 3, 5, 2, 1, 0\} \tag{1}$$

   (a) Determine the Lloyd-Max quantizer with $R = 1$ bit / symbol. The initial intervals are $I_1 = (-\infty, 2]$ and $I_2 = (2, +\infty]$.

   (b) Encode the sequence showing the decoded symbols and the corresponding MSE.

   (c) Compute the empirical entropy of the quantization symbols.

   (d) Compute the prediction residuals in open loop. Based on this result, compute the Lloyd-Max quantizer. The initial intervals are $I_1 = (-\infty, 0]$ and $I_2 = (0, +\infty]$.

   (e) Encode the sequence with DPCM showing the decoded symbols and the corresponding MSE (assume that the first symbol is lossless coded).

   Solution

   (a) Lloyd-Max iterates the following steps

      1. Given the thresholds, update the reconstruction level

$$y_1 = (2 + 1 + 1 + 2 + 1 + 0)/6 = 7/6 = 1.167 \tag{2}$$
$$y_2 = (3 + 5)/2 = 4 \tag{3}$$

      2. Given the reconstruction level, update the thresholds. That is, $a_2 = (y_1 + y_2)/2 = 2.5833$
      3. Given the thresholds, update the reconstruction level

$$y_1 = (2 + 1 + 1 + 2 + 1 + 0)/6 = 7/6 = 1.167 \tag{4}$$
$$y_2 = (3 + 5)/2 = 4 \tag{5}$$

      4. Convergence.

   (b) The decoded sequence is $\{7/6, 7/6, 7/6, 4, 4, 7/6, 7/6, 7/6\}$. The MSE is

$$MSE = \frac{1}{8}[(2-7/6)^2+(1-7/6)^2+(1-7/6)^2+(3-4)^2+(5-4)^2+(2-7/6)^2+(1-7/6)^2+(0-7/6)^2] = 0.6042 \tag{6}$$

   (c) $p(y_1) = 6/8$, $p(y_2) = 2/8$. The entropy is $H(X) = 0.8113$ bits / symbol.

   (d) The prediction residuals are $-, -1, 0, 2, 2, -3, -1, -1$. Lloyd-Max iterates the following steps

1. Given the thresholds, update the reconstruction level

$$y_1 = (-1 + 0 - 3 - 1 - 1)/5 = -6/5 \tag{7}$$
$$y_2 = (2 + 2)/2 = 2 \tag{8}$$
$$\tag{9}$$

2. Given the reconstruction level, update the thresholds. That is, $a_2 = (y_1 + y_2)/2 = 0.4$
3. Given the thresholds, update the reconstruction level

$$y_1 = (-1 + 0 - 3 - 1 - 1)/5 = -6/5 \tag{10}$$
$$y_2 = (2 + 2)/2 = 2 \tag{11}$$
$$\tag{12}$$

4. Convergence.

(e) DPCM coding proceeds as follows

x	2	1	1	3	5	2	1	0
e	-	-1	0.2	3.4	3.4	-1.6	-1.4	-1.2
$\tilde{e}$	-	-1.2	-1.2	2	2	-1.2	-1.2	-1.2
$\tilde{x}$	2	0.8	-0.4	1.6	3.6	2.4	1.2	0
$x - \tilde{x}$	0	0.2	1.4	1.4	1.4	-0.4	-0.2	0

The MSE is equal to 0.765.

2. **Question (5 pts)**
   (a) Illustrate Elias coding for the case of a binary memoryless source.
   (b) Describe how Elias coding can be adapted to take into account time-varying statistics.

   Solution

   See slide set `MMSP_1.1_Lossless_Coding`.

3. **Question (5 pts)**
   (a) Illustrate the block diagram of an image coding system. Briefly describe the input and output of each block.
   (b) Comment on the computational complexity of the different blocks.

   Solution

   See slide set `MMSP_2.1_Image_Coding`.

4. **Questions (5 pts - each answer can be either TRUE or FALSE)**

   (a) Consider a transform coding system, where T is the $M \times N$ transform matrix.

   - [T] [F] When $M = N$, the transform is critically sampled.
     TRUE. When the number of transform coefficients equals the number of input samples, the transform is called critically sampled
   - [T] [F] Let $M = N$. $\sum_{i=1}^{N} \sigma_{x_i} = \sum_{i=1}^{N} \sigma_{y_i}$
     FALSE. $\sum_{i=1}^{N} \sigma_{x_i}^2 = \sum_{i=1}^{N} \sigma_{y_i}^2$
   - [T] [F] Let R_i denote the rate allocated to the ith coefficient. Under optimal bit allocation, $R_i = R + c \log_2 \sigma_{y_i}^2$
     TRUE. The rate is proportional to the logarithm of the variance of the coefficient.
   - [T] [F] The KLT is the transform that minimizes $\frac{1}{N} \sum_{i=1}^{N} \sigma_{y_i}$.
     FALSE. The KLT maximizes the coding gain. Hence, it maximizes $\left(\prod_{i=1}^{N} \sigma_{y_i}^2 \right)^{1/N}$

   (b) Consider an audio coding system

   - [T] [F] For some critical bands, it might happen that SMR is negative.
     TRUE. Some maskers (e.g. noise maskers) are able to mask maskees having higher energy.
   - [T] [F] Bit allocation is inversely proportional to the SMR
     FALSE. It is directly proportional.
   - [T] [F] The MDCT achieves perfect reconstruction.
     TRUE.

   (c) Let X denote a discrete source with memory.

   - [T] [F] It is possible to design a code whose average length $L < H(X)$
     TRUE. L is lower bounded by the entropy rate $H_\infty(X)$, which is less than $H(X)$ for a source with memory.
   - [T] [F] $H(x_1, \ldots, x_N)/N \geq H(x_N | x_{N-1}, \ldots, x_1)$
     TRUE. Conditional encoding, which requires at least $H(x_N | x_{N-1}, \ldots, x_1)$ bits / symbol, is more efficient than blockwise encoding (which requires at least $H(x_1, \ldots, x_N)/N$ bits/ symbol).
   - [T] [F] $L_H \leq L_S$, where L_S and L_H denote, respectively, the average length of Shannon and Huffman coding.
     TRUE. Huffman coding is optimal among all prefix codes. Hence, $L_H \leq L_S$

5. **Question (Matlab - 10pts)**

   Load the dataset `data.m`.

   (a) Let $\{z_i\}_{i=1}^{M}$ denote the sequence of symbols to be encoded. Let $\{x_i\}_{i=1}^{M/2}$ and $\{y_i\}_{i=1}^{M/2}$ denote, respectively, the sequence of symbols at odd and even time instants. Plot the samples in the $x - y$ plane.

   (b) Perform scalar quantization with $R = 1$ bits / symbol. Design the optimal quantizer for both $\{x_i\}_{i=1}^{M/2}$ and $\{y_i\}_{i=1}^{M/2}$.

   (c) Perform optimal vector quantization with $R = 1$ bits / symbols, coding 2-dimensional vectors $[x_i, y_i]^T$, $i = 1, \ldots, M/2$.

   (d) Compare the SNR in the two cases.

   (e) Plot the reconstruction levels in both cases, superimposed to the data samples on a 2-dimensional plane.

Solution

```
load('data');

M = length(z);

x = z(1:2:end);
y = z(2:2:end);

plot(x,y,'co')

%% L-M for x
maxiter = 1000;
ax = 0;
MSEx = Inf;
for i = 1:maxiter
    x1 = mean(x(x <= ax));
    x2 = mean(x(x  > ax));
    ax = (x1+x2)/2;

    temp = (sum((x(x <= ax) - x1).^2) + sum((x(x > ax) - x2).^2))/(M/2);
    if abs(temp - MSEx) > 1e-4
        MSEx = temp;
    else
        break
    end
end

SNRx = 10*log10(var(x) / MSEx(end));

%% L-M for y
maxiter = 1000;
ay = 0;
MSEy = Inf;
for i = 1:maxiter

    y1 = mean(y(y <= ay));
    y2 = mean(y(y  > ay));
    ay = (y1+y2)/2;

    temp = (sum((y(y <= ay) - y1).^2) + sum((y(y > ay) - y2).^2))/(M/2);
    if abs(temp - MSEy) > 1e-4
        MSEy = temp;
    else
        break
    end
end

SNRy = 10*log10(var(y) / MSEy(end));

SNRxy = 10*log10(var(z) / mean( [MSEx(end), MSEy(end)] ));

%% Vector quantization
R = 1;
N = 2;
L = 2^(N*R);

% initialization
cw = [x(1:L),y(1:L)]';
MSEvq = Inf;
zin = [x, y]';
zq = NaN*ones(N,M/N);
label = NaN*ones(1, M/N);

for iter = 1:maxiter

    for i = 1:M/N
        dist = Inf;
        for l = 1:L

            temp = norm(zin(:,i) - cw(:,l));
            if temp < dist
                dist = temp;
                label(i) = l;
                zq(:,i) = cw(:,l);
            end
        end
    end

    for l = 1:L
        cw(:,l) = mean(zin(:,label == l), 2);
    end

    temp = sum(sum((zin - zq).^2)) / M;
    if abs(temp - MSEvq) > 1e-4
        MSEvq = temp;
    else
        break
    end
end
```

```
SNRvq = 10*log10(var(z) / MSEvq);


%% display
hold on
plot(cw(1,:), cw(2,:), 'k*');

plot(x1, y1, 'bd');
plot(x1, y2, 'bd');
plot(x2, y1, 'bd');
plot(x2, y2, 'bd');

hold off
```